

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Piña Camacho	19/11/2025
	Nombre: Javier	

Actividad 2. Generación de entornos de pruebas y casos de test asistidos por IA

Esta actividad se centrará en la creación, validación y mejora de los testeos en una aplicación generada con IA. Se toma como base la aplicación generada en la actividad 1. La aplicación consiste en un punto de venta para una farmacia que permite el registro de cada venta ligándola al empleado que la realizó.

La aplicación con el código generado tanto para la actividad 1 como para la actividad 2 se puede ver en este enlace: <https://github.com/Javier-P-C/Generacion de codigo con IA UNIR>

Funcionalidades de la Aplicación

Para determinar el tipo de pruebas que se necesitan es necesario conocer todo lo que la aplicación debe ser capaz de lograr. A continuación, se enlistan cada una de las funciones del punto de venta:

- Autenticación
 - Login
 - Refresh
 - Permisos por rol
 - Rutas protegidas en frontend
- Empleados
 - Listar empleados
 - Crear empleado
 - Editar empleado
 - Eliminar empleado
 - Validación de roles
- Productos
 - Listar productos
 - Crear producto
 - Editar producto
 - Eliminar producto
 - Validar campos
- Inventario
 - Listar inventario
 - Aumentar inventario (solo manager)
 - Validación de stock ≥ 0
- Ventas
 - Registrar venta con múltiples productos
 - Validación de stock por cada elemento

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Piña Camacho	19/11/2025
	Nombre: Javier	

- Calcular subtotal por elemento
- Calcular total
- Disminuir inventario
- Listar ventas
- Ver detalle de venta
- Frontend
 - Carga de todas las pantallas
 - Validaciones visuales para las acciones del usuario
 - Errores y mensajes

Pruebas Pre Creadas

Al crear la aplicación en la actividad 1 se usó un prompt generado con IA, en él se pidieron pruebas unitarias para la aplicación. Al inicio del proyecto se creó una descripción de las especificaciones de la aplicación deseadas. Esta descripción fue mejorada con ayuda de ChatGPT para crear un prompt óptimo para Copilot. El prompt mejorado incluía la petición de buenas prácticas para la creación del código, una de estas prácticas fue la inclusión de pruebas unitarias.

Las pruebas se crearon para el backend, para enlistarlas le pedí a Copilot que hiciera un resumen de ellas.

Enlista los tests que ya tiene este proyecto dentro de su código

backend

frontend

README.md

> Used 3 references

Basándome en los archivos del proyecto, aquí están los tests que ya tiene implementados:

Tests Existentes en el Proyecto

1. employees/tests.py

Tests del módulo de empleados (usuarios):

- `test_create_employee`: Verifica la creación de un empleado básico
- `test_create_manager`: Verifica la creación de un empleado con rol de Manager
- `test_create_assistant`: Verifica la creación de un empleado con rol de Asistente
- `test_employee_str`: Verifica la representación en string del modelo Employee
- `test_default_role`: Verifica que el rol por defecto sea ASSISTANT

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Piña Camacho	19/11/2025
	Nombre: Javier	

2. products/tests.py

Tests del módulo de productos:

- `test_create_product`: Verifica la creación de un producto
- `test_product_str`: Verifica la representación en string del modelo Product
- `test_presentation_choices`: Verifica las opciones válidas de presentación
- `test_price_validation`: Verifica que el precio no pueda ser negativo

3. inventory/tests.py

Tests del módulo de inventario:

- `test_inventory_created_on_product_creation`: Verifica que se cree automáticamente un registro de inventario al crear un producto
- `test_inventory_str`: Verifica la representación en string del modelo Inventory
- `test_quantity_cannot_be_negative`: Verifica que la cantidad no pueda ser negativa
- `test_increase_inventory`: Verifica el aumento de inventario
- `test_decrease_inventory`: Verifica la disminución de inventario

4. sales/tests.py

Tests del módulo de ventas (el más completo):

- `test_create_sale_with_details`: Verifica la creación de una venta con detalles
- `test_sale_updates_inventory`: Verifica que la venta actualice el inventario
- `test_sale_calculates_total`: Verifica el cálculo automático del total
- `test_cannot_sell_more_than_inventory`: Verifica que no se pueda vender más de lo disponible
- `test_sale_detail_str`: Verifica la representación en string de SaleDetail
- `test_multiple_items_sale`: Verifica la venta de múltiples productos
- `test_insufficient_stock`: Verifica el manejo de stock insuficiente
- `test_sale_with_zero_quantity`: Verifica que no se permita cantidad cero

Resumen por Categoría

Módulo	Número de Tests	Cobertura Principal
employees	5	Creación de usuarios y roles
products	4	Gestión de productos y validaciones
inventory	5	Control de stock y señales
sales	8	Proceso de ventas y reglas de negocio
TOTAL	22 tests	Cobertura de funcionalidades críticas

Pruebas Unitarias

En base a las funcionalidades de la aplicación se determinaron que las siguientes pruebas unitarias deben crearse para la validación de la aplicación. Se le pedirá al agente de Copilot que añada las pruebas faltantes, si es que hacen falta, para cumplir con el siguiente listado.

- Autenticación
 - Probar la generación correcta del token JWT en el login.
 - Validar que la recarga de token devuelve un nuevo token de acceso
 - Validar que el manager puede acceder a endpoints restringidos.
 - Validar que los asistentes reciben error en endpoints de administración.

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Piña Camacho	19/11/2025
	Nombre: Javier	

- Empleados
 - Crear empleado
 - Editar empleado
 - Eliminar empleado
- Productos
 - Listar productos
 - Crear producto:
 - Validar presentación válida.
 - Validar precio decimal correcto.
 - Editar producto
 - Eliminar producto
- Inventario
 - Listar inventario
 - Aumentar inventario (solo Manager)
 - Evitar inventario negativo al disminuir stock
 - Validación de cantidades no negativas
- Ventas
 - Crear venta con múltiples productos
 - Validación de stock suficiente por elemento
 - Calcular precio subtotal de un producto
 - Calcular precio total de varios productos
 - Verificar que el inventario se reduce correctamente.
 - Los empleados pueden listar las ventas
 - Los empleados pueden ver el detalle de las ventas
- Frontend
 - Render de cada componente individual sin error
 - Validación de formularios (sin llamar al backend)
 - Mostrar mensajes de error

Pruebas de Integración

Las pruebas de integración que se decidieron pedir para generar la aplicación fueron las siguientes:

- Autenticación
 - Login completo: enviar credenciales, recibir token, almacenar en frontend
 - Recarga de token automático
 - Probar que usuario autenticado pueda acceder a la aplicación
 - Probar que usuario no autenticado no pueda acceder a la aplicación
 - Probar que los asistentes no pueden acceder a rutas del manager
- Empleados
 - Crear empleado desde la UI y revisar que aparezca en la tabla

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Piña Camacho	19/11/2025
	Nombre: Javier	

- Editar empleado desde la UI
- Productos
 - Crear un producto desde la UI y checar que aparece en la tabla
 - Editar producto desde la UI
 - Eliminar producto y desaparece de la lista.
 - Intentar crear o editar producto con datos inválidos y mostrar errores del backend en UI
- Inventario
 - Mostrar lista completa de inventario cargada desde backend
 - Aumentar inventario como Manager y stock actualizado en UI
 - Intentar aumentar inventario como Assistant y debe bloquearse
 - Registrar venta y verificar
 - inventario baja automáticamente
 - cambios reflejados en frontend
- Ventas
 - Registrar venta desde UI con múltiples productos
 - Verificar que el backend devuelve total y subtotales correctos
 - Intentar vender más de lo disponible y detectar error visible en frontend
 - Listado de ventas cargado correctamente
 - Validar que cuando el stock es insuficiente se recibe mensaje de error
- Frontend general
 - Carga de todas las pantallas tras login
 - Mensajes de éxito al crear/editar
 - Manejo de expiración de token con logout automático o recarga.

Resumen de Pruebas Generadas

Para generar los tests mencionados previamente se mandó un prompt a Copilot con las mismas listas de este documento y se le pidió que añadiera los tests necesarios sin borrar con los que ya contaba el proyecto.

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Piña Camacho	19/11/2025
	Nombre: Javier	

Prompt (El resto del mensaje es un copia y pega de las listas de las pruebas mencionadas anteriormente):

Quiero que añadas los cambios necesarios para contar con las siguientes pruebas unitarios y de integración dentro del proyecto. Si es necesario puedes añadir dependencias para poder ejecutar las pruebas. De igual forma actualiza el README con los pasos necesarios para ejecutarlas. No modifiques como funciona el proyecto solo encárgate de construir pruebas.

Las pruebas unitarias que se quieren son:

Autenticación

- Probar la generación correcta del token JWT en el login.
- Validar que la recarga de token devuelve un nuevo token de acceso
- Validar que el manager puede acceder a endpoints restringidos.
- Validar que los asistentes reciben error en endpoints de administración.

Empleados

- Crear empleado
- Editar empleado

Después de trabajar en los cambios Copilot respondió con un resumen de todo el trabajo.

 **¡Pruebas Añadidas Exitosamente!**

He añadido **135+ pruebas unitarias y de integración** completas al proyecto, cumpliendo con todos los requisitos solicitados.

 **Resumen de lo Implementado**

✅ Pruebas Backend (100+ tests)

Archivos Creados/Modificados:

1.  tests.py (NUEVO) - 8 pruebas de autenticación JWT
2.  tests.py (EXPANDIDO) - 11 pruebas de empleados
3.  tests.py (EXPANDIDO) - 15 pruebas de productos
4.  tests.py (EXPANDIDO) - 13 pruebas de inventario
5.  tests.py (EXPANDIDO) - 16 pruebas de ventas
6.  integration_tests.py (NUEVO) - 20+ pruebas de integración

✅ Pruebas Frontend (35+ tests)

Archivos Creados:

7.  components.test.jsx - 5 pruebas de componentes
8.  pages.test.jsx - 11 pruebas de páginas
9.  auth.spec.js - 6 pruebas E2E de autenticación
10.  integration.spec.js - 14+ pruebas E2E de integración

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Piña Camacho	19/11/2025
	Nombre: Javier	

 Configuración

Archivos Actualizados:

- `package.json` - Dependencias de testing añadidas (vitest, @testing-library, playwright)
- `vitest.config.js` - Configuración de Vitest
- `playwright.config.js` - Configuración de Playwright
- `setup.js` - Setup de pruebas

 Scripts de Ejecución

Archivos Creados:

- `run-tests.sh` - Script Bash para ejecutar todas las pruebas
- `run-tests.ps1` - Script PowerShell para ejecutar todas las pruebas

 Documentación Completa

Archivos Creados:

- `TESTING.md` - Documentación completa de pruebas (detallada)
- `QUICK_TEST_GUIDE.md` - Guía rápida paso a paso
- `TEST_SUMMARY.md` - Resumen de cambios
- `TEST_CHECKLIST.md` - Checklist de verificación
- `README.md` - Actualizado con sección de testing

 Cobertura de Requisitos

Backend Unit Tests 

-  **Autenticación:** JWT, refresh token, permisos manager/assistant
-  **Empleados:** Crear, editar, eliminar
-  **Productos:** Listar, crear con presentación válida, precio decimal, editar, eliminar
-  **Inventario:** Listar, aumentar (Manager), evitar negativo, validaciones
-  **Ventas:** Múltiples productos, validar stock, calcular precios, reducir inventario, listar/ver detalles

Backend Integration Tests 

-  Login completo con almacenamiento de token
-  Recarga automática de token
-  Usuario autenticado/no autenticado
-  Asistente bloqueado de rutas manager
-  CRUD completo desde UI
-  Proceso completo de venta con actualización de inventario

Frontend Unit Tests 

-  Render de componentes sin error
-  Validación de formularios (sin backend)
-  Mostrar mensajes de error

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Piña Camacho	19/11/2025
	Nombre: Javier	

Frontend E2E Tests

-  Login completo con credenciales y token
-  Recarga automática de token
-  Control de acceso autenticado/no autenticado
-  Asistente bloqueado de rutas manager
-  Crear/editar empleados y productos desde UI
-  Eliminar productos con verificación
-  Datos inválidos muestran errores
-  Lista completa de inventario
-  Aumentar inventario como manager
-  Registrar venta y verificar reducción de inventario
-  Cálculos correctos de totales
-  Manejo de stock insuficiente
-  Todas las pantallas cargan después del login
-  Mensajes de éxito en operaciones

Documentación Disponible

Documento	Propósito
 README.md	Overview general con sección de testing
 TESTING.md	Documentación completa y detallada de todas las pruebas
 QUICK_TEST_GUIDE.md	Guía rápida paso a paso para ejecutar pruebas
 TEST_SUMMARY.md	Resumen de todos los cambios implementados
 TEST_CHECKLIST.md	Checklist de verificación

Garantías

-  **No se modificó la funcionalidad existente** - Solo se añadieron pruebas
-  **100% de los requisitos cubiertos** - Todas las pruebas solicitadas están implementadas
-  **Documentación completa** - Múltiples guías y referencias
-  **Scripts automatizados** - Ejecutar todas las pruebas con un comando
-  **Listo para CI/CD** - Las pruebas pueden integrarse en pipelines
-  **Fácil de mantener** - Estructura clara y bien documentada

¡Todo listo para usar! 

Al correr las pruebas creadas algunas fallaron. Cada que sucedía se copiaron los logs a Copilot directamente en el chat y el agente los corrigió uno por uno hasta que la ejecución fue completamente exitosa.

Ejecución

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Piña Camacho	19/11/2025
	Nombre: Javier	

Copilot creo dos scripts para ejecutar las pruebas de manera automática, uno es para Linux y otro para powershell.

Para que funcione el script se tienen que hacer dos cosas antes. Primero se deben instalar las dependencias locales d npm:

- En la terminal localizate en la raíz del proyecto y ejecuta los siguientes comandos:
 - cd frontend/
 - npm install

Lo segundo es asegurarse de que se levante la aplicación. Los scripts de las pruebas esperan 90 segundos para que el docker compose levante todo y empiezan con las pruebas. Es probable que no sea tiempo suficiente para levantar toda la aplicación en la computadora por eso recomiendo correrla en una terminal aparte y luego correr el script de las pruebas en otra terminal.

- Ve a la raíz del proyecto y ejecuta el siguiente comando o su equivalente en tu ambiente:
 - docker compose up --build

Una vez hecho lo anterior se puede correr el script de las pruebas.

Si se va a correr en Linux o WSL se debe usar:

```
chmod +x run-tests.sh
./run-tests.sh
```

Si se va a correr en Powershell se debe ejecutar:

```
.\run-tests.ps1
```

Si al ejecutar el script salen errores de Playwright como el siguiente, instala el browser necesario en la carpeta de frontend con el comando que menciona el error.

```
20) [chromium] > e2e/integration.spec.js:264:3 > Frontend General Tests > success messages on create/edit operations

Error: browserType.launch: Executable doesn't exist at /home/javier/.cache/ms-playwright/chromium_headless_shell-1194/chrome-linux/headless_shell

Looks like Playwright Test or Playwright was just installed or updated.
Please run the following command to download new browsers:

  npx playwright install

<3 Playwright Team
```

También se pueden ejecutar las pruebas individualmente. Para ello el agente creo una guía en el archivo QUICK_TEST_GUIDE.md.

Logs

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Piña Camacho	19/11/2025
	Nombre: Javier	

Se mostrarán los siguientes logs que marcan el inicio de cada sección de pruebas. Si alguna de las secciones falla el script parará y no continuará con la siguiente sección.

```

acción en Desarrollo de Software/Generacion_de_codigo_con_IA_UNIR$ ./run-tests.sh
=====
Pharmacy Management System - Test Runner
=====

=====
1. Running Backend Unit Tests
=====
→Testing: Authentication, Employees, Products, Inventory, Sales
→Ensuring Docker services are up
WARN[0000] /mnt/d/javie/Documents/DevOps_Master/Generación de Código y Automatización en Desarrollo de Software/Generacion_de_codigo_con_IA_UNIR/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
Creating test database for alias 'default' ('test_pharmacy_db')...
Found 65 test(s).
Operations to perform:
  Synchronize unmigrated apps: authapp, corsheaders, messages, rest_framework, staticfiles
  Apply all migrations: admin, auth, contenttypes, employees, inventory, products, sales, sessions
Synchronizing apps without migrations:
  Creating tables...
  Running deferred SQL...
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
=====
2. Running Backend Integration Tests
=====
→Testing: End-to-end workflows
WARN[0000] /mnt/d/javie/Documents/DevOps_Master/Generación de Código y Automatización en Desarrollo de Software/Generacion_de_codigo_con_IA_UNIR/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
Creating test database for alias 'default' ('test_pharmacy_db')...
Found 19 test(s).
Operations to perform:
  Synchronize unmigrated apps: authapp, corsheaders, messages, rest_framework, staticfiles
  Apply all migrations: admin, auth, contenttypes, employees, inventory, products, sales, sessions
Synchronizing apps without migrations:
  Creating tables...
  Running deferred SQL...
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0001_initial... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK

```

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Piña Camacho	19/11/2025
	Nombre: Javier	

```

=====
3. Running Frontend Unit Tests
=====
→Testing: Components and Pages

> frontend@1.0.0 test
> vitest --run

RUN v1.6.1 /mnt/d/javie/Documents/DevOps_Master/Generación de Código y Automatización en D
esarrollo de Software/Generacion_de_codigo_con_IA_UNIR/frontend

=====
4. Running Frontend E2E Tests
=====
→Testing: Complete user journeys
→Note: This requires the application to be running

> frontend@1.0.0 e2e
> playwright test

Running 20 tests using 6 workers
 20 passed (13.5s)

To open last HTML report run:

  npx playwright show-report

✓ Frontend E2E tests passed

=====
✓ All Tests Passed Successfully!
=====

```

Conclusiones

El uso de la inteligencia artificial en la creación de pruebas es una de las implementaciones más útiles en el desarrollo de software. La creación de pruebas es una actividad tediosa y repetitiva que a pesar de agregar valor a cada proyecto también lo alenta. En esta actividad se crearon exitosamente pruebas de integración, pruebas unitarias e incluso pruebas de interfaz por medio de IA. Usar esta capacidad en cada proyecto es sin duda una buena manera de agilizar el proceso de mantenimiento.

Para hacer las pruebas adecuadas siempre se requerirá de una guía humana que comprenda el alcance del proyecto. Es por eso que inicialmente se plantearon las pruebas pertinentes a las funcionalidades y después de determinarlas se le entregaron a la IA. Con el contexto completo del proyecto y los objetivos delimitados por el dueño del proyecto la IA es capaz de desarrollar todo lo necesario para las pruebas, sobre todo con el modo agente, como fue en este caso.

Asignatura	Datos del alumno	Fecha
Generalización de Código de Automatización en Desarrollo de Software con IA	Apellidos: Piña Camacho	19/11/2025
	Nombre: Javier	

El mayor problema que tuvo la IA fue con las pruebas de interfaz, lo cual tiene sentido porque depende del análisis de cada ejecución para validar que los botones y textos que busca realmente aparecen. En proyectos profesionales sería mejor monitorear o crear directamente este tipo de pruebas manualmente. El resto de las pruebas que dependen de la lógica del código pueden ser creadas completamente por IA hoy en día. Sin embargo, es importante tener transparencia en el código teniendo a desarrolladores capaces de entender cada cambio nuevo generado por la IA.